Journal of Automation and Intelligence 4 (2025) 299–311

Contents lists available at ScienceDirect

Journal of Automation and Intelligence

journal homepage: www.keaipublishing.com/en/journals/journal-of-automation-and-intelligence/

Research article

# Novel multi-agent action masked deep reinforcement learning for general industrial assembly lines balancing problems

Ali M. Ali, Luca Tirel, Hashim A. Hashim *

*Department of Mechanical and Aerospace Engineering, Carleton University, Ottawa, ON, K1S-5B6, Canada*

ARTICLE INFO

*Keywords:*
Artificial intelligence in industrial engineering
Autonomous decision making
Distributed multi-agent learning
Reinforcement learning

ABSTRACT

Efficient planning of activities is essential for modern industrial assembly lines to uphold manufacturing standards, prevent project constraint violations, and achieve cost-effective operations. While exact solutions to such challenges can be obtained through Integer Programming (IP), the dependence of the search space on input parameters often makes IP computationally infeasible for large-scale scenarios. Heuristic methods, such as Genetic Algorithms, can also be applied, but they frequently produce suboptimal solutions in extensive cases. This paper introduces a novel mathematical model of a generic industrial assembly line formulated as a Markov Decision Process (MDP), without imposing assumptions on the type of assembly line a notable distinction from most existing models. The proposed model is employed to create a virtual environment for training Deep Reinforcement Learning (DRL) agents to optimize task and resource scheduling. To enhance the efficiency of agent training, the paper proposes two innovative tools. The first is an action-masking technique, which ensures the agent selects only feasible actions, thereby reducing training time. The second is a multi-agent approach, where each workstation is managed by an individual agent, as a result, the state and action spaces were reduced. A centralized training framework with decentralized execution is adopted, offering a scalable learning architecture for optimizing industrial assembly lines. This framework allows the agents to learn offline and subsequently provide real-time solutions during operations by leveraging a neural network that maps the current factory state to the optimal action. The effectiveness of the proposed scheme is validated through numerical simulations, demonstrating significantly faster convergence to the optimal solution compared to a comparable model-based approach.

## 1. Introduction

The growing complexity of activities in large, modern industries necessitates advanced planning systems for manufacturing and assembly lines to optimize key performance indicators while adhering to specific operational constraints. Assembly Line Balancing Problems (ALBPs) are well-established, challenging nonlinear programming problems characterized by context-specific formulations and significant computational demands [1]. During the execution phase, unforeseen disruptions such as equipment breakdowns can derail planned schedules, and the time-intensive process of recalculating solutions may render real-time adjustments impractical, leading to project delays and substantial unplanned costs. Effective assembly line balancing directly influences productivity, throughput, and cost-efficiency, making it a critical factor for industries striving to maintain a competitive edge [2]. In sectors with complex production processes, such as automotive and electronics manufacturing, ALBP plays a pivotal role in ensuring timely delivery, maintaining quality control, and responding efficiently to customer demands, thus driving profitability and operational excellence [1]. ALBP can be conceptualized as a control problem centered on the efficient assignment of tasks and resources to satisfy constraints while optimizing key performance metrics [3]. This conceptualization allows ALBP to be addressed through reinforcement learning (RL) approaches, where agents learn to allocate tasks and resources on the assembly line to maximize an objective function. Recent studies [4–8] have successfully applied Deep Reinforcement Learning (DRL) frameworks to various allocation and control problems across fields such as blockchain, energy demand management, humanoid robotics, and traffic signal control. These advancements highlight DRL as a promising approach for effectively addressing the challenges associated with assembly line balancing.

Peer review under responsibility of Chongqing University.
* Corresponding author.
*E-mail addresses:* alimohamedali@cmail.carleton.ca (A.M. Ali), luca.tirel@cmail.carleton.ca (L. Tirel), hhashim@carleton.ca (H.A. Hashim).

https://doi.org/10.1016/j.jai.2025.07.001
Received 16 February 2025; Received in revised form 13 July 2025; Accepted 18 July 2025
Available online 22 July 2025
2949-8554/© 2025 The Authors. Published by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

中国知网 https://www.cnki.net

**Nomenclature**

**Symbols**

| | |
|---|---|
| $i$ | Subscript indicating workstations |
| $j$ | Subscript indicating tasks |
| $r$ | Subscript indicating resources |
| $z$ | Subscript indicating a specific action in the actions space |
| $k$ | Discrete time instant |
| $\boldsymbol{I}$ | Set of workstations |
| $\boldsymbol{J}$ | Set of tasks |
| $\boldsymbol{R}$ | Set of resources |
| $\delta_s$ | Dimension of the state |
| $\delta_A$ | Dimension of the centralized actions space |
| $\delta_{A_i}$ | Dimension of the decentralized actions space |
| $\boldsymbol{O}$ | Workstations' capacities in terms of tasks |
| $\boldsymbol{U}$ | Workstations' capacities in terms of resources |
| $\boldsymbol{D}$ | Tasks' durations workstation-dependent |
| $\boldsymbol{F}$ | Tasks' deadlines time instants |
| $\boldsymbol{P}$ | Tasks' time precedence constraints matrix |
| $\boldsymbol{C}$ | Tasks' required amount of resources |
| $\boldsymbol{G}$ | Resources of each type available in the assembly line |
| $\boldsymbol{A}$ | Centralized tasks actions space |
| $\boldsymbol{A}_i$ | Decentralized tasks actions space |
| $\boldsymbol{A}_s[k]$ | Subset of feasible actions at time $k$ |
| $\boldsymbol{a}[k]$ | Centralized control action for task assignment |
| $\boldsymbol{y}[k]$ | Centralized control action for resources assignment |
| $\boldsymbol{a}_i[k]$ | Decentralized control action for task assignment |
| $\boldsymbol{o}[k]$ | Workstations' states of occupancies |
| $\boldsymbol{e}[k]$ | Tasks' states of execution |
| $\boldsymbol{f}[k]$ | Tasks' states of finish |
| $\boldsymbol{d}[k]$ | Tasks' current remaining durations |
| $\boldsymbol{g}[k]$ | Resources levels in the assembly line |
| $\boldsymbol{r}[k]$ | Resources stored for the tasks executions |
| $\boldsymbol{b}[k]$ | Resources currently stored in a specific workstation |
| $\boldsymbol{m}[k]$ | Current centralized action mask |
| $\boldsymbol{m}_i[k]$ | Current decentralized action mask |
| $\boldsymbol{h}[k]$ | Ending state flag |
| $\boldsymbol{s}[k]$ | State of the factory at time $k$ |
| $k_{\text{end}}$ | Episodic ending time |
| $k_{\text{opt}}$ | Best achievable ending time computed via O.C. |
| $\mathcal{E}$ | Environment |
| $\mathcal{A}_i$ | Agent $i$th |
| $\boldsymbol{p}$ | Action probabilities |
| $\boldsymbol{p}_{\text{f}}$ | Action probabilities filtered based on the mask |
| $f_{\text{learn}}$ | Learning frequency of the PPO |
| $\boldsymbol{s}^{\text{F}}$ | Fictitious state |
| $\boldsymbol{m}^{\text{F}}$ | Fictitious actions mask |

## 2. Related work

### 2.1. Optimal control in ALBPs

An assembly line consists of multiple workstations, each tasked with performing specific operations on items within a designated time interval, known as the cycle time. Solving the ALBP requires the optimal assignment of tasks to workstations while ensuring compliance with predefined constraints. Various formulations of ALBP are explored in Ref. [9], which categorize the problem into two primary types: the Single Assembly Line Balancing Problem (SALBP) and the more comprehensive Generalized Assembly Line Balancing Problem (GALBP).

The work presented in Ref. [3] represents an early attempt to address ALBPs. Two primary families of ALBP problems were identified in Ref. [3]: the SALBP and the GALBP. The SALBP is characterized by single-element production, a serial line layout, a fixed common cycle time, a serial unilateral line, precedence constraints, and deterministic operation times. The GALBP formulation relaxes some of the assumptions of the SALBP to accommodate more realistic scenarios. Specifically, GALBPs allow for the consideration of multi-model processes, zone constraints, delays, parallel stations, and more complex layouts, among other factors. This broader class of problems includes the U-shaped Assembly Line Balancing Problem (UALBP) and the Mixed-Model Assembly Line Balancing Problem (MMALBP) [10]. Another classification of ALBP is based on the objective function [9]. Type-1 focuses on minimizing the number of workstations required to meet a fixed finishing time while optimizing resource utilization and maintaining production rates. In contrast, Type-2 aims to minimize the finishing time for a given number of workstations. Exact solutions to the SALBP are typically derived by repeatedly solving instances of the closely related SALBP-1. For instance, Ref. [11] proposes a two-level approach, where the first level estimates production lot sizes, while the second level focuses on rescheduling planned tasks. In Ref. [12], the authors developed an Integer Programming (IP) model to optimize operational planning for Printed Circuit Board (PCB) manufacturing. Historically, methods such as linear programming, integer programming, dynamic programming, and branch-and-bound have been employed to address ALBPs. However, these traditional techniques have increasingly been supplanted by search heuristics, such as genetic algorithms, which offer enhanced computational efficiency, as demonstrated in Refs. [13–16]. Recent studies have also focused on improving the robustness of these solutions. For example, Ref. [17] formulated a mixed-integer programming model to optimally allocate jobs and workers to workstations, leveraging historical data to reliably estimate optimal capacities. Additionally, Ref. [18] presented an integrated approach for designing reconfigurable assembly systems, accounting for potential future demand scenarios to mitigate reconfiguration costs.

### 2.2. Deep reinforcement learning for ALBPs

A key advantage of DRL methodologies is their adaptability. Once a model is trained to address a specific problem, solutions do not need to be recomputed in the event of unforeseen disruptions. Instead, agents can be retrained or fine-tuned for new scenarios, providing substantial flexibility [19]. However, a common challenge with Reinforcement Learning (RL) techniques is that they may yield solutions of lower quality compared to traditional methods, which has led to the development of hybrid approaches. For example, Ref. [20] introduced a hybrid approach that combines an RL-based scheduling algorithm with classical Optimal Control (OC). In Ref. [21], the authors proposed a smart scheduling agent based on a DRL algorithm, which demonstrated robustness in handling uncertainties such as unexpected task arrivals or equipment malfunctions. Similarly, Ref. [22] addressed an industrial scheduling problem under uncertain conditions by utilizing a DRL-based Proximal Policy Optimization (PPO) algorithm.



中国知网 https://www.cnki.net

This study will further explore advanced variants of both the on-policy PPO algorithm [23] and the off-policy Deep Q-Network (DQN) algorithm [24] to solve a Fully Observable Markov Decision Process (FOMDP). More recently, Ref. [25] examined optimal maintenance planning for scheduling in a serial production line, showcasing the applicability of DRL to complex industrial scheduling problems.

### 2.3. Action masking in reinforcement learning

Classical optimal approaches define the solution space through constraints, with feasible solutions identified within this space. In contrast, in RL, satisfying constraints may lead to infeasible state transitions, requiring careful action selection to ensure feasibility. This can be addressed using a state-dependent function that maps the action space, distinguishing feasible actions from infeasible ones. One of the earliest efforts to extend the Markov Decision Process (MDP) with a feedback mechanism to reject invalid actions is presented in Ref. [26]. This framework introduced "shielding", a technique that monitors an agent's actions and corrects them if they violate specified safety properties. In Ref. [27], the authors explored action masking within policy gradient algorithms, defining and comparing various strategies for handling invalid actions and offering valuable theoretical insights. Notably, they demonstrated that a state-dependent action masker can function as a differentiable component, thereby enabling valid policy gradient updates. Building on this work, Ref. [28] extended these principles to a multi-agent framework. They synthesized multiple shields based on a factorization of the joint state space across agents, with each shield monitoring a subset of agents, ensuring constraint adherence at every time step. Both action masking and shielding techniques have proven effective in eliminating unsafe and infeasible states during training. Action masking filters the action space to allow only feasible actions through a state-dependent function while shielding monitors the agent's actions and corrects them only when they violate predefined specifications. In the proposed work, action masking technique is employed to manage the numerous constraints inherent in the assembly line balancing problem. Without this approach, the large number of constraints could result in many futile actions, thereby increasing training time [27].

### 2.4. Multi agent reinforcement learning

The primary concept in Multi-Agent Reinforcement Learning (MARL) lies in decentralizing a single learning entity into multiple agents [29]. While the overall complexity of MARL schemes can vary significantly depending on the domain, the core idea involves each agent learning its own policy so that, collectively, all agents achieve the system's overarching goal. An extensive review of different types of MARL is provided in Ref. [30], where the authors categorize tasks as cooperative, competitive, or general (neither purely cooperative nor competitive) and highlight a key challenge in this field: the formal articulation of a shared learning objective. Specifically, MARL algorithms must be designed to avoid complete independence from other agents or mere behavioral tracking, both of which could hinder convergence. A significant contribution to MARL theory was made in Ref. [31], where the authors developed a multi-agent Q-learning framework and proved convergence to a Nash equilibrium under certain conditions. Their findings demonstrated that convergence requires only that each action is attempted and each state is visited, enabling each agent to learn optimal Q-values without assuming specific behaviors of other agents, provided the immediate rewards of other agents are observable. In Ref. [32], the authors addressed cooperative multi-agent control using DRL, introducing parameter sharing among agents' policies. They extended algorithms such as DQN, Deep Deterministic Policy Gradient (DDPG), and Trust Region Policy Optimization (TRPO) to multi-agent settings, thereby ensuring scalability with continuous action spaces. Additional effective value-based approaches in MARL were proposed in Refs. [33,34]. Practical recommendations for implementing MARL policy gradient algorithms, aimed at achieving optimal convergence, were provided in Ref. [35]. Furthermore, Ref. [36] identified PPO as an effective algorithm for cooperative multi-agent games. In Ref. [37], the authors proposed a multi-agent centralized policy gradient method with non-monotonic factorization, capable of handling both discrete and continuous action spaces.

### 2.5. Paper contributions

Motivated by the excessive computational burden of solving the ALBP using exact optimal control, DRL presents an alternative solution to address the problem. Once sufficiently trained in a simulated environment, DRL agents can provide real-time solutions by leveraging the forward pass of the neural network policy. Previous research has primarily focused on the ALBP in centralized settings, often tailored to specific types of assembly lines. Given the typically lengthy training times required for DRL, a generalized learning framework is necessary one that can adapt to diverse assembly line configurations without the need for retraining for each variation. Furthermore, techniques aimed at reducing training time would greatly enhance the efficiency and scalability of the training process. The main contributions of this work can be summarized as follows:

- Unlike Ref. [38], this paper proposes a mathematical formulation of the GALBP within a DRL context, where a centralized environment interacts with multiple agents, rather than focusing solely on a serial assembly line.
- The decentralization challenge is addressed by transitioning from a single-agent to a multi-agent formulation, incorporating a sequential framework to enforce constraint satisfaction.
- To reduce training time, an action mask is employed to filter infeasible actions. This action mask is defined for both single-agent and multi-agent frameworks through a sequential feasibility check.
- A comparative analysis is conducted between DRL-trained agents and the OC approach, which serves as the baseline for evaluation.

*Structure.* The remainder of the paper is organized into five sections. Section 3 presents the problem formulation. A brief description of the proposed masking technique is discussed in Section 4. Section 5 demonstrates the proposed decentralization strategy. Section 6 illustrates the effectiveness of the proposed multi-agent scheme through numerical simulations. Finally, Section 7 concludes the work.

## 3. Problem formulation

The mathematical formulation of the factory model requires defining state variables, the action space, a set of constraints, and the design of an optimization objective. Our focus will be on the problem formulation within the MDP framework. Specifically, a finite MDP is defined by the tuple $(S, A, p, \rho)$, where $S$ represents the set of states, $A$ denotes the set of actions, and $p(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s}')$ indicates the state-transition probability, which describes the likelihood that taking action $\boldsymbol{a}[k]$ from state $\boldsymbol{s}[k]$ leads to state $\boldsymbol{s}[k+1]$ (or $\boldsymbol{s}'$). The symbol $\rho$ represents the immediate reward received after the state transition resulting from taking action $\boldsymbol{a}$.

The RL scheduling problem must be formulated as a constrained MDP. Given the current state of the factory at time $k$, denoted $\boldsymbol{s}[k]$, not all actions are feasible, only those that satisfy a specific set of constraints. Therefore, a subset $A_s[k] \subseteq A$ of the action space is admissible to ensure safe behavior. This necessitates the definition of a masking function $\mathcal{M}(s)$, which computes the admissible actions at a given state. The use of the masking function guarantees constraint satisfaction. In both proposed algorithms, our goal is to learn (via a neural network) the policy $\pi_{\theta_k}(s)$ that maximizes the expected cumulative reward, where $\theta_k$ represents the weights of the neural network at



中国知网 https://www.cnki.net

time $k$. In the case of on-policy PPO, we also need to learn a state-value function, which is approximated by a neural network with parameters $\phi_k$.

### 3.1. Parameters and state variables

We denote the sets of positive integers and binary numbers as $\mathbb{Z}+$ and $\mathbb{Z}_2$, respectively, with $|.|$ representing the cardinality of a set. The key parameters are as follows: the time horizon is $|K| \in \mathbb{Z}+$, the number of workstations is $|I| \in \mathbb{Z}+$, the number of tasks is $|J| \in \mathbb{Z}+$, and the number of different types of resources involved in the problem is $|R| \in \mathbb{Z}+$. The parameters related to the constraints are $\boldsymbol{O} \in \mathbb{Z}+^{|I|}$ and $\boldsymbol{U} \in \mathbb{Z}+^{|I|\times|R|}$, which represent the maximum capacities of the workstations in terms of the number of simultaneous tasks that can be processed and the amount of resources that can be stored, respectively. With respect to task constraints, $\boldsymbol{P} \in \mathbb{Z}_2^{|J|\times|J|}$ is a Boolean matrix representing time precedence constraints, $\boldsymbol{F} \in \mathbb{Z}+^{|J|}$ specifies the deadlines for each task, $\boldsymbol{D} \in \mathbb{Z}+^{|I|\times|J|}$ represents the task durations (which depend on the workstations), and $\boldsymbol{C} \in \mathbb{Z}+^{|J|\times|R|}$ indicates the amount of each specific resource needed for task completion. Finally, $\boldsymbol{G} \in \mathbb{Z}+^{|R|}$ represents the overall availability of each resource type in the factory. The primary state variables of interest, which, when concatenated and flattened, represent the state of the factory, include the number of tasks currently being processed at the workstations (referred to as "occupancies") and denoted by $\boldsymbol{o}[k] \in \mathbb{Z}+^{|I|}$, the remaining durations for task completion, denoted $\boldsymbol{d}[k] \in \mathbb{Z}+^{|I|\times|J|}$, and the resources currently stored for task execution, denoted by $\boldsymbol{r}[k] \in \mathbb{Z}+^{|I|\times|J|\times|R|}$. Other auxiliary variables are utilized to model the transitions of tasks between starting, execution, and completion states, as well as the availability of resources in both the factory and workstation storage. While additional features can be incorporated into the state vector when necessary, such additions would increase the vector's size and require larger neural networks for the agents, thereby significantly increasing the training computation time. Therefore, careful consideration is required when adding new features to ensure they are essential for capturing the dynamics of the factory. The resulting state vector, formed by concatenation, has the following dimensionality: $\delta_s = \dim(\boldsymbol{s}[k]) = |I| \cdot (1 + |J| \cdot (1 + |R|))$.

### 3.2. Factory dynamics

At the beginning of the simulation, unless the system is initialized in an intermediate state with pre-assigned tasks, the factory state is represented as a vector with $\delta_s$ zero entries. At time $k-1$, when the agent chooses to assign a task $j$ to workstation $i$, the corresponding $i$th entry of $\boldsymbol{o}[k]$ is incremented by one, and this increment continues until the task is completed:

$$\boldsymbol{o}[k+1](i) = \begin{cases} \boldsymbol{o}[k](i) + 1 & \text{if } \boldsymbol{a}[k](i,j) = 1 \\ \boldsymbol{o}[k](i) & \text{otherwise} \end{cases} \tag{1}$$

The $(i,j)$-th entry of $\boldsymbol{d}[k]$ is then assigned to the corresponding initial duration value taken from $D$:

$$\boldsymbol{d}[k+1](i,j) = \begin{cases} D(i,j) & \text{if } \boldsymbol{a}[k](i,j) = 1 \\ \boldsymbol{d}[k](i,j) - 1 & \text{if } \boldsymbol{e}[k](i,j) = 1 \\ \boldsymbol{d}[k](i,j) & \text{otherwise} \end{cases} \tag{2}$$

After the assignment, at each time step, the duration is decreased by one, transitioning tasks from the execution state $\boldsymbol{e}[k] \in \mathbb{Z}_2^{|I|\times|J|}$ to the finishing state $\boldsymbol{f}[k]$. To minimize the search space, the resource action is kept binary, while the resources allocated for task execution, represented by the variable $\boldsymbol{r}[k]$, are integer-valued. This is enforced through the following dynamic constraint:

$$\boldsymbol{r}[k+1](i,j,r) = \begin{cases} \boldsymbol{r}[k](i,j,r) + C(j,r) & \text{if } \boldsymbol{y}[k](i,j,r) = 1 \\ \boldsymbol{r}[k](i,j,r) - C(j,r) & \text{if } \boldsymbol{d}[k](i,j) = 0 \\ \boldsymbol{r}[k](i,j,r) & \text{otherwise} \end{cases} \tag{3}$$

The resource assignment also corresponds to a reduction in factory inventories (i.e. $G[k+1](r) = G[k](r) - C(j,r)$). The resources assigned are considered task-specific since this improves traceability and allows modeling resources that do not vanish with consumption, such as tools.

### 3.3. Actions space

To solve the scheduling problem, dynamic assignment of tasks and resources to the workstations is required. These assignments define the actions within the problem. Let $\boldsymbol{a}[k] \in \mathbb{Z}_2^{|I|\times|J|}$ represent a Boolean matrix, where the $(i,j)$ entry equals one if task $j$ is assigned to workstation $i$ at time $k$. Similarly, let $\boldsymbol{y}[k] \in \mathbb{Z}_2^{|I|\times|J|\times|R|}$ denote a Boolean tensor, where the entry corresponding to resource $r$ and task $j$ at workstation $i$ at time $k$ is one if resource $r$ is assigned to task $j$ at workstation $i$. It is evident that the total number of possible actions, denoted as $\delta_A = |A|$, grows exponentially with the problem size.

$$\delta_A = 2^{|I|\times|J|} \tag{4}$$

The constraints inherent in the problem limit the number of feasible actions. Specifically, a task can be assigned to only one workstation at a time, and similarly, each resource can be allocated to a single task at a workstation. This imposes a summation constraint of one along the workstation dimension of the tensor. The number of actions that satisfy this unique assignment constraint is:

$$\delta_A = (|I| + 1)^{|J|} \tag{5}$$

In fact, each task can be either assigned to one workstation or not assigned at all. The number of feasible actions in the centralized framework further reduces after taking into account the fact that the workstations have a limited capacity to handle multiple simultaneous tasks, whose total number can at most be equal to the specific workstation's maximal occupancy. It can be proved that the number of actions satisfying both the unique assignment constraint and the workstation occupancy constraint is given by:

$$\begin{aligned} \delta_A = \sum_{i_1=0}^{O_1} \binom{|J|}{i_1} \sum_{i_2=0}^{\min\{O_2,|J|-i_1\}} \binom{|J|-i_1}{i_2} \\ \sum_{i_3=0}^{\min\{O_3,|J|-i_1-i_2\}} \binom{|J|-i_1-i_2}{i_3} \\ \sum_{i_{|I|}=0}^{\min\{O_{|I|},|J|-i_1-\cdots-i_{|I|-1}\}} \binom{|J|-i_1-\cdots-i_{|I|-1}}{i_{|I|}} \end{aligned} \tag{6}$$

where $O_i$ represents the $i$th value of simultaneous occupancies limiting vector $O$. This initial dimension is further reduced in the next step by introducing a decentralized framework. Furthermore, we simplify the problem by assuming that tasks and resource assignments are interlinked (i.e., if the $(i,j)$ component of $\boldsymbol{a}[k]$ equals one, then the $(i,j,r)$ components of the resource action $\boldsymbol{y}[k]$ will also equal one, $\forall r$. Fig. 1 depicts the factory dynamics for a simple case of 3 workstations and 5 tasks. At time zero, the task allocation control matrix $\boldsymbol{a}[0]$ assigns tasks 1 and 3 to workstation 1, while workstation 2 is not assigned any tasks, as indicated by the second row being zero in $\boldsymbol{a}[0]$. Workstation 3 is assigned to task 5. In each subsequent time step, the remaining duration $\boldsymbol{d}[1]$ decreases by one. Additionally, the resource inventory for workstation 1 and workstation 3 decreases accordingly. The dynamics of resource allocation follow a similar pattern, in alignment with the resource allocation matrix for each type of resource. At this specific time instant, the resource allocation matrix $\boldsymbol{y}[k]$ is expected to assign resources to workstation 1 and workstation 3 in subsequent time steps, assuming the agent is effectively learning to manage factory resources. The control scheme for the assembly line is shown in Fig. 2.



中国知网 https://www.cnki.net

### 3.4. Constraints and actions masks

The primary constraints in the problem include task deadlines, limited resource and storage availability, and precedence relationships among tasks. Additionally, logical constraints such as the inability to reassign completed tasks and the requirement for correct transitions between task states must also be enforced. Given the concatenated factory state $s[k]$ and the previously outlined constraints, an action masking function maps the current state to a binary vector $\boldsymbol{m}[k]$ of the same dimension as the action space. In this vector, 0 represents infeasible actions, and 1 indicates feasible actions. The action masking function is formally defined as follows:

$$\mathcal{M}(s) : \mathbb{Z}+^{\delta_s} \longrightarrow \mathbb{Z}_2^{\delta_A} \tag{7}$$

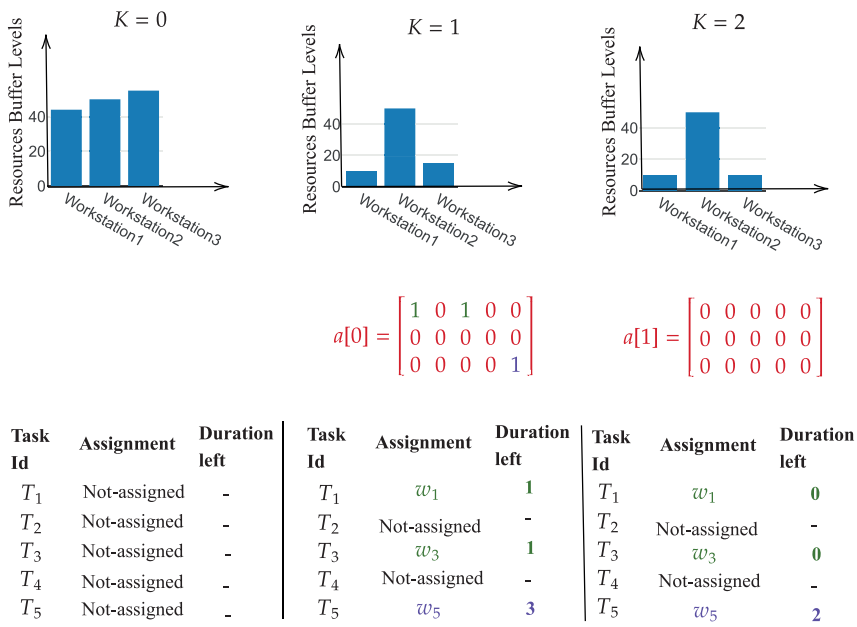



**Fig. 1.** The visualization of the factory dynamics for a scenario involving 3 workstations and 5 tasks.

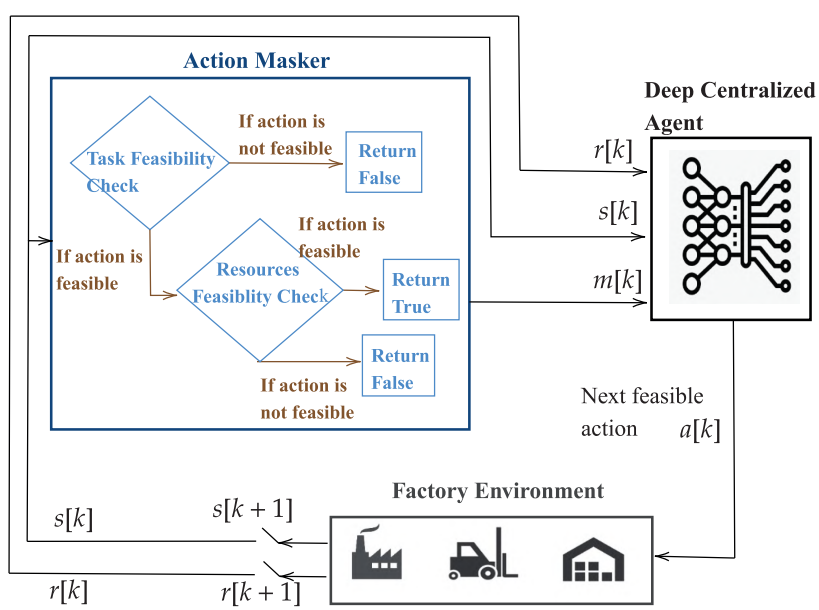



**Fig. 2.** The central agent architecture which introduces a modified version of the DQN/PPO agents making use of the action mask concept.



中国知网 https://www.cnki.net

where the set of constraints used to do the mapping is defined in detail in Appendix.

### 3.5. Reward function

Formally defining an RL problem necessitates characterizing the reward function. In this work, we propose the following reward function:

$$\rho(\boldsymbol{a}, \boldsymbol{s})[k] = \beta \cdot \frac{\boldsymbol{h}[k]}{1 + k^{\alpha}}. \tag{8}$$

where $\alpha$ and $\beta$ are positive scalar design parameters, and $\boldsymbol{h}[k]$ is a Boolean flag indicating whether all tasks have finished executing. The motivation behind the proposed reward function in (8) is to assign a higher reward for completing all tasks within fewer time steps, denoted by $k$. The absence of a reward in many episodes is not problematic, particularly when a fixed time horizon is considered, as the objective in reinforcement learning is to maximize the expected cumulative reward.

## 4. Proposed masking technique

Action masking involves blocking any infeasible action suggested by the neural network agent and replacing it with the null action. As will be shown in the simulation section, action masking can significantly speed up the learning process. In this work, we have developed an action masking technique for both the off-policy DQN and the on-policy PPO algorithms, with only minor differences in their implementations. The process of action masking at the network's output is shown in Fig. 3. The details are explained in the following subsections.

### 4.1. DQN action masking

This paper presents a modified version of the DQN from Ref. [39], specifically adapted to incorporate action masking. In this approach, the policy network, which receives the centralized state as input, outputs $\delta_A$ Q-values. Prior to selecting the action with the highest Q-value, all Q-values corresponding to infeasible actions are set to negative infinity. As a result, the agent always selects feasible actions, even when a random action is chosen for exploration according to the well-known $\epsilon$-greedy policy. The masking technique also affects the loss computation, particularly when calculating the loss between predicted and target Q-values. In this process, the Q-values associated with infeasible actions are masked by setting them to $-\infty$.

### 4.2. PPO action masking

The PPO algorithm from Ref. [23] has been adapted in this work to integrate an action masking technique. The training process of a PPO agent involves the definition of two networks: an actor (or policy) network and a value network. The outputs of the actor network represent the probabilities for each possible action, given the current input state. In this approach, these probabilities are filtered by the action mask, which sets the probabilities corresponding to infeasible actions to negative infinity. The masked action probabilities are then re-normalized using the Softmax layer at the output, which is essential for multi-class classification tasks. As a result, infeasible actions are assigned null probabilities, while the probabilities of feasible actions are normalized so that they sum to one, as demonstrated in Ref. [27]. During the cost function optimization, the updated action probabilities, after applying the masking, are taken into account. One of the key advantages of using policy gradient methods with action masking is the stronger guarantee of convergence compared to action-value approximation. Specifically, the gradient update steps remain smooth, whereas action-value selection can lead to non-smooth gradient updates.

### 4.3. Action masking effects

The adoption of the mask drastically improves the convergence of the algorithm to the optimal policy. To show that, a PPO agent has been trained in the same environment and provided with a negative reward for taking infeasible actions. As an example, Fig. 4 shows the comparison of the episodic length (finishing time of all tasks) between a masked-PPO and the classical one, highlighting the effectiveness of the strategy. In the latter case, there is no convergence to the optimal policy, whereas the masked version achieves an optimal solution within only 400 training episodes.

## 5. Problem decentralization strategy and proposed series constraints enforcement scheme

The dependence of the observation and action space dimensions on factory parameters, and the resultant dimension explosion, must be considered to maintain scalability. Bearing this in mind, RL possesses a suitable structure for learner decentralization, thereby reducing computational complexity. In our scenario, the idea is to decentralize the assignment at the level of the workstations. We can consider each workstation as an agent aiming to solve a cooperative task. The constraints satisfaction plays an important role; in fact, logic must be implemented to guarantee that the actions selected by the agents are not in conflict with each other. Therefore, a more robust control logic

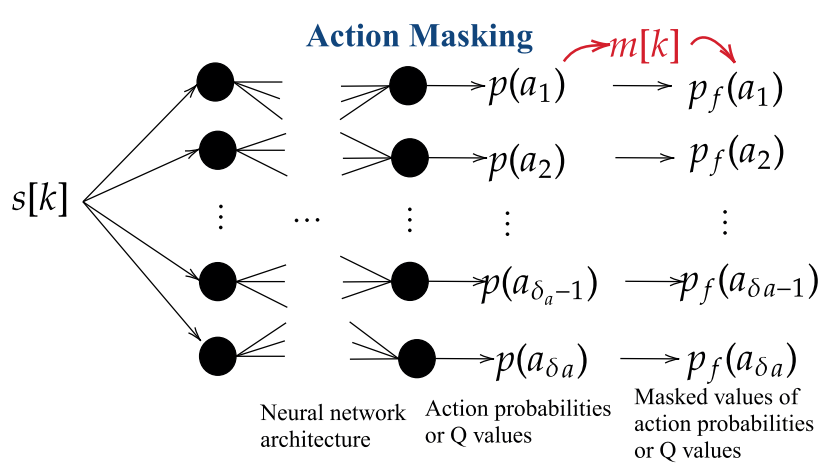



**Fig. 3.** Action masking at the network output.



中国知网 https://www.cnki.net

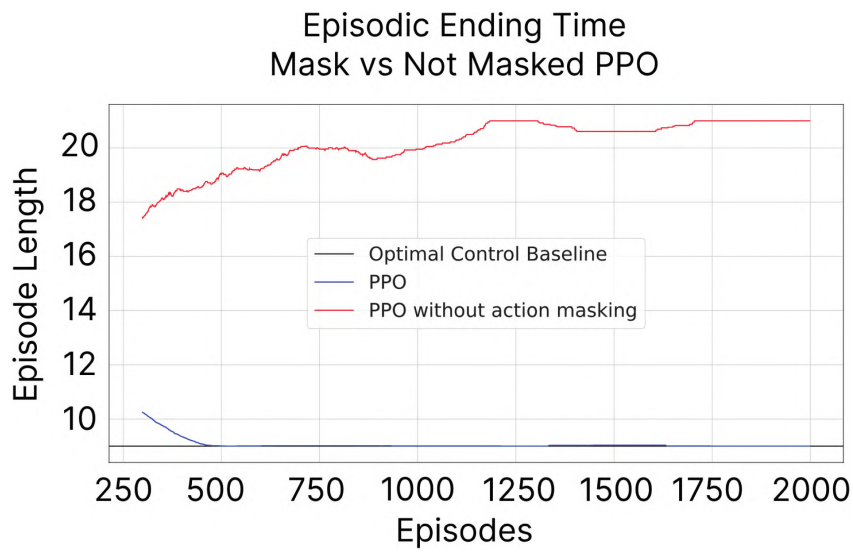



**Fig. 4.** Action masking effect on the episodic length (finishing time of all tasks) for centralized PPO algorithms compared to an equivalent optimal control problem in [40].

is required to efficiently ensure that the concatenated action, resulting from the decentralized agents' choices, is feasible. This problem has been solved through a sequential feasibility check that makes use of the masks.

### 5.1. Multi-agent formulation

The multi-agent problem requires decentralization of the observations space, the actions space, the actions masking, and the design of a new reward function. The assumption is made that the controlled system is fully observable, which is reasonable in the case of a factory. The centralized fully observable state will be therefore maintained. The action space has been decentralized by considering the specific occupancy constraint for the $i$th workstation. This imposes a summation constraint on each action, ensuring that workstations cannot assign themselves more tasks than their occupancy bounds allow. The following decentralized dimensions are obtained:

$$\delta_{A_i} = \sum_{j=0}^{O_i} \binom{|J|}{j}, \quad \forall i \tag{9}$$

This implies that the $|I|$ decentralized mappings will have varying co-domain dimensions:

$$\mathcal{M}_i(\boldsymbol{s}) : \mathbb{Z}+^{\delta_s} \longrightarrow \mathbb{Z}_2^{\delta_{A_i}}, \quad \forall i \tag{10}$$

The centralized action for task assignment $\boldsymbol{a}[k]$ can be redefined by splitting it row-wise. Each row consists of task assignments to a specific workstation, enabling us to redefine each row as the decentralized action $\boldsymbol{a}_i[k]$. The reward function employed for this problem is consistent with the one used in the centralized case, and it is shared among all agents. Furthermore, all agents have access to the same observations.

### 5.2. Sequential feasibility check

Sequential actions taken independently by the agents might result in an infeasible aggregated action. For this reason, the same initial state is fed to all the agents, outputting their predicted actions. Then these actions are concatenated sequentially into a matrix with the same shape as the centralized action, and its feasibility is checked. If the action is not feasible, it is surely due to a conflict between the agents' choices, necessitating a feasibility check. This is done by iteratively selecting a random agent and applying its proposed action to a simulated copy of the environment. Then the resulting new simulated state is evaluated. The fictitious state $\boldsymbol{s}^{\mathrm{F}}$, is then applied as input to the next agent in the sequence, which will compute the next best action to be applied, based on a mask computed from the fictitious state, defined as $\boldsymbol{m}^{\mathrm{F}}$. This loop will proceed sequentially for all the $|I|$ agents, outputting a feasible concatenated action. The main idea of this loop is to provide the agents with a copy of the real state of the factory, where assets are allocated following agents' choices but without decreasing the duration of the tasks (e.g., without executing them). Based on this fictitious state, a mask is computed. The mask can be considered a fictitious mask because it is computed dependently on the choices of the previous agents in the loop. (e.g., if the first agent in the loop assigns a task, the fictitious mask $\boldsymbol{m}^{\mathrm{F}}$ computed by the next agent will not see that task as an assignable one, even if it is). To prevent biases, the order is randomized during learning. The use of this control loop allows the agents to be "conscious" of the behavior of the others, guaranteeing an efficient action selection in a decentralized way. Agents in this loop are randomly selected during training to avoid biased results. The sequential feasibility check control scheme is shown in Fig. 5. The sequential feasibility check procedure is detailed in Algorithm 1, in which the following functions are defined:

- **InitFakeEnv**($\boldsymbol{s}^{\mathrm{F}}[k], \boldsymbol{m}^{\mathrm{F}}[k]$): this function is used to initialize the fictitious environment.
- **FakeTransition**($\boldsymbol{a}[k], \boldsymbol{s}^{\mathrm{F}}[k], \mathcal{E}^{\mathrm{F}}$): it is used to update the current simulated state and masks as a result of the sequential actions proposed by each agent. These transitions of the simulated states are used to check the feasibility of the proposed concatenated action, computed sequentially (they do not cause a real state transition).

The dependencies from time $k$ will be omitted in the following algorithm for the sake of clarity (e.g., $\boldsymbol{s}[k]$ and $\boldsymbol{m}[k]$ will be treated as $\boldsymbol{s}$ and $\boldsymbol{m}$).

### 5.3. Multi agent training loop

The pseudocode for the MARL training loop is presented in Algorithm 2. The main functions defined and used are:

- **Reset**($\mathcal{E}$): it is the reset function of the environment. It returns the episodic initial state.



中国知网 https://www.cnki.net

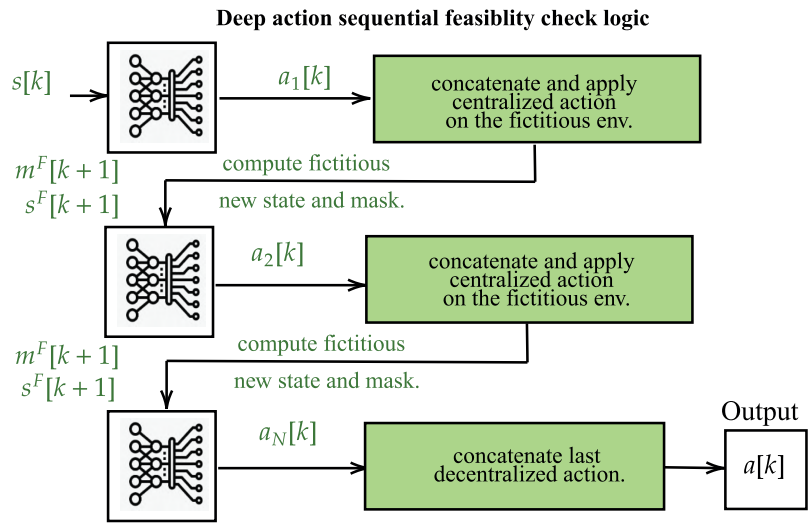



**Fig. 5.** Sequential action feasibility check control logic.

- **GetAgentMask**$(\boldsymbol{s}[k], \mathcal{A}_i, \mathcal{E})$: is called on the environment to get a mask depending on the current state, for each agent.
- **GetAgentAction**$(\boldsymbol{s}[k], \boldsymbol{m}_i[k], \mathcal{A}_i)$: this function is called on each agent to output the predicted action using the current policy.
- **GetStateValue**$(\boldsymbol{s}[k], \mathcal{A}_i)$: is called on each agent to get the value function for the given state, for each agent.
- **SFC**$(\boldsymbol{s}[k], \boldsymbol{a}[k], \boldsymbol{m}[k])$: is the main control logic used to create a feasible concatenated action once we detect an infeasibility source, applying Algorithm 1.
- **Transition**$(\boldsymbol{a}[k], \mathcal{E})$ is the state transition function of the environment.
- **ComputeProbsNewAction**$(\boldsymbol{s}[k], \boldsymbol{a}_i[k], \boldsymbol{m}_i[k], \mathcal{A}_i)$: it is used if we change an infeasible action proposed by the agent to a safe one. It computes the probabilities of the new safe action for the memory update.
- **MemoUpdate** $(\boldsymbol{s}[k], \boldsymbol{a}_i[k], \boldsymbol{p}_i[k], \boldsymbol{v}_i[k], \boldsymbol{r}[k+1], \boldsymbol{h}[k+1], \boldsymbol{m}_i[k])$: This function is used to update the memory buffer of each agent and collect safe state trajectories using the mask embedding. Here, $\boldsymbol{p}_i[k]$ refers to a specific element of $\boldsymbol{p}_f$ for the $i$-th agent, depending on the selected action $\boldsymbol{a}_i[k]$.
- **MaskedPPOLearningStep**$(\mathcal{A}_i)$: indicates the learning steps of Masked PPO-Clip.

**Algorithm 1** Sequential Feasibility Check (Randomized)

**Require:** Environment $\mathcal{E}$, Agents $\mathcal{A}_i$
**Require:** Current State $\boldsymbol{s}[k]$
**Require:** Current Mask $\boldsymbol{m} = [\boldsymbol{m}_1, \dots, \boldsymbol{m}_{\delta_{\mathcal{A}_i}}]$
**Require:** Initial Feasible Action $\boldsymbol{a} = [\dots, 0, \dots]$
1: $\boldsymbol{s}^{\mathrm{F}} = \boldsymbol{s}$, $\boldsymbol{m}^{\mathrm{F}} = \boldsymbol{m}$
2: $\mathcal{E}^{\mathrm{F}} = \mathrm{InitFakeEnv}(\boldsymbol{s}^{\mathrm{F}}, \boldsymbol{m}^{\mathrm{F}})$
3: **for** $i = 1, \dots, |I|$ **do**
4: Choose a random agent $\mathcal{A}_i$
5: $\boldsymbol{a}_i = \mathrm{GetAgentAction}(\boldsymbol{s}^{\mathrm{F}}, \boldsymbol{m}_i^{\mathrm{F}}, \mathcal{A}_i)$
6: $\boldsymbol{a} = [\dots, \boldsymbol{a}_i, \dots]$
7: $\boldsymbol{s}^{\mathrm{F}}, \boldsymbol{m}^{\mathrm{F}} = \mathrm{FakeTransition}(\boldsymbol{a}, \boldsymbol{s}^{\mathrm{F}}, \mathcal{E}^{\mathrm{F}})$
8: **end for**
9: **return** $\boldsymbol{a}[k]$

Algorithm 2 shows the training loop for the multi-agent system. The time dependencies have been omitted for the sake of clarity. It starts by initializing the environment of the factory and the number of agents, whereas for each episode it starts by resetting the environment, and

**Algorithm 2** Multi-agent PPO Training Loop.

**Require:** Environment $\mathcal{E}$
**Require:** PPO Agents $\mathcal{A}_i$, for $i \in 1, \dots, |I|$
1: **for** $n_{\text{episodes}} = 1, \dots, N_{\text{episodes}}$ **do**
2: $\boldsymbol{s} = \mathrm{Reset}(\mathcal{E})$
3: **while** not done **do**
4: **for** $i = 1, \dots, |I|$ **do**
5: $\boldsymbol{m}_i = \mathrm{GetAgentMask}(\boldsymbol{s}, \mathcal{A}_i, \mathcal{E})$
6: $\boldsymbol{a}_i[k] = \mathrm{GetAgentAction}(\boldsymbol{s}, \boldsymbol{m}_i, \mathcal{A}_i)$
7: $\boldsymbol{v}_i = \mathrm{GetStateValue}(\boldsymbol{s}, \mathcal{A}_i)$
8: **end for**
9: $\boldsymbol{a} = [\boldsymbol{a}_1, \dots, \boldsymbol{a}_{|I|}]$
10: $\boldsymbol{m} = [\boldsymbol{m}_1, \dots, \boldsymbol{m}_{|I|}]$
11: **if** $\boldsymbol{a}$ is not feasible **then**
12: $\boldsymbol{a} = \mathrm{SFC}(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{m})$
13: **end if**
14: $\boldsymbol{s}'$, $\boldsymbol{r}$, $\boldsymbol{h} = \mathrm{Transition}(\boldsymbol{a}, \mathcal{E})$
15: **for** $i = 1, \dots, |I|$ **do**
16: $\boldsymbol{p}_i = \mathrm{ComputeProbsNewAction}(\boldsymbol{s}, \boldsymbol{a}_i, \boldsymbol{m}_i, \mathcal{A}_i)$
17: $\mathrm{MemoUpdate}(\boldsymbol{s}, \boldsymbol{a}_i, \boldsymbol{p}_i, \boldsymbol{v}_i, \boldsymbol{r}, \boldsymbol{h}, \boldsymbol{m}_{i,t})$
18: **end for**
19: **if** $f_{\text{learn}} = \mathrm{True}$ **then**
20: $\mathrm{MaskedPPOLearningStep}(\mathcal{A}_i), \forall i$
21: **end if**
22: $\boldsymbol{s} \leftarrow \boldsymbol{s}'$
23: **end while**
24: **end for**

while not all the tasks are finished it proceeds with the main while loop. In the main while Loop in step 3 for each agent, the algorithm computes the current action mask and builds the concatenated action based on the policy of each agent. Then in step 11, the feasibility of the concatenated action is checked, and if it is not feasible, a feasible action is created in step 12. In step 14, the feasible concatenated action is applied to the environment. $\boldsymbol{s}'$ is a simplified notation for $\boldsymbol{s}[k+1]$. Subsequently, the memory is updated. Learning, based on the masked version of PPO, continues until all tasks are completed. For DQN, the sole modification to Algorithm 2 starts from step 15, where learning is based on the masked version of DQN rather than PPO. The proposed framework work falls under centralized training and decentralized execution within a multi-agent framework. Centralized



中国知网 https://www.cnki.net

training and decentralized execution have been shown to effectively address challenges in wireless power allocation [41] and power control in heterogeneous networks [42]. The primary reason for adopting this strategy in the proposed work is the implementation of a sequential feasibility check, which determines whether an action at a given workstation is feasible. This feasibility check must be centralized to ensure, for instance, that a task has not already been assigned by another workstation.

## 6. Results

This section presents the main results and proposes some empirical tuning guidelines. All simulations were conducted on an Intel(R) Core(TM) i7-10750H CPU @ 2.60 GHz, 2592 MHz, 6 cores, 16 GB RAM, and a 64-bit processor running Windows 11. The key parameters of the factory and the agents are summarized in Tables 1, 2, and 3. These parameters were fine-tuned through multiple experiments to identify the optimal configurations. Special attention should be given to tuning the target synchronization frequency in DQN and the learning frequency in PPO. Although a grid search over the parameters could be conducted for further optimization, this was deferred to future work due to computational constraints. Empirically, good performance was observed when the network's input layer size was set to 3 to 5 times the input size $\delta_s$. In general, PPO exhibited less sensitivity to hyperparameters compared to DQN. For both PPO and DQN, the update frequency is the most crucial parameter, as it influences both the stability and efficiency of the learning process. The remaining parameters in Table 1 are relatively straightforward to tune. A smaller value for the learning frequency and target synchronization frequency than those specified in Table 1 may lead to more frequent updates between the Q-network and the target network, which can cause instability in the learning process. Conversely, larger values can enhance stability but may result in slower learning improvements.

### 6.1. Agents comparison

The training processes of the agents were compared, with the solution to the problem formulated within the optimal control (OC) framework serving as a benchmark for performance evaluation. The agents were tuned to converge to the same outcomes as the equivalent OC problem. The training time for the PPO agent was found to be the shortest overall, even with a larger network. Additionally, the decentralization of the problem justifies the use of a smaller network size, as it leads to smaller action spaces and, consequently, reduces training time by up to 50%. In Fig. 6, the episodic completion times achieved during training for both the centralized and decentralized versions of DQN and PPO are depicted, with the solution obtained from OC shown in black. The $y$-axis represents the achieved $k_{\text{end}}$ in each episode, where $k_{\text{end}}$ denotes the time instance when all tasks are completed, or when the environment reaches the horizon value. The objective in DRL is to achieve the minimum possible ending time $k_{\text{opt}}$, which is computed by solving an optimization problem.

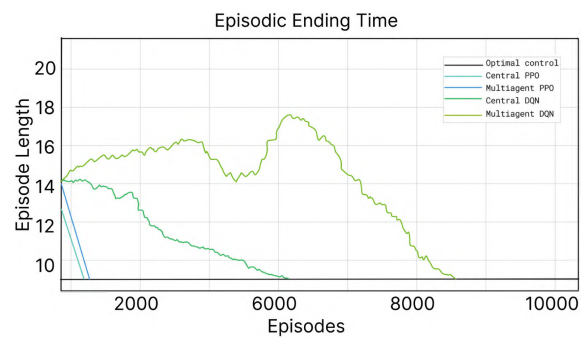



**Fig. 6.** Comparison of the episodic ending times (time necessary to complete all tasks) during training.

### 6.2. Robustness empirical test to a random initialization of the trained PPO agent

To evaluate the robustness of the proposed model, an empirical test was conducted. In this test, a pre-trained PPO agent was initialized with a set of random initial states. The minimum achievable ending time

**Table 1**
Agent parameters.

| Hyperparameters | DQN Centralized | DQN multi-agent | PPO Centralized | PPO multi-agent |
|---|---|---|---|---|
| Learning Rate (BOTH) | 1e−5 | 1e−5 | 3e−4 | 3e−4 |
| Batch Size (BOTH) | 64 | 64 | 5 | 5 |
| Discount (BOTH) | 0.995 | 0.995 | 0.99 | 0.99 |
| Memory (BOTH) | 100 000 | 100 000 | 100 000 | 100 000 |
| Target Sync Frequency (DQN) | 10 | 10 | – | – |
| Soft Update Weight (DQN) | 0.8 | 0.8 | – | – |
| Gradient Clipping Value (DQN) | 1 | 1 | – | – |
| Learning Warm start (DQN) | 200 | 200 | – | – |
| Likelihood Ratio Clipping (PPO) | – | – | 0.2 | 0.2 |
| Learning Frequency (PPO) | – | – | 20 | 20 |
| Learning Epochs (PPO) | – | – | 4 | 4 |
| GAE lambda value (PPO) | – | – | 0.95 | 0.95 |

**Table 2**
Networks parameters.

| Network Shape | DQN | DQN Multi-agent | PPO | PPO Multi-agent |
|---|---|---|---|---|
| Input Layer Neurons | $\delta_s$ | $\delta_s$ | $\delta_s$ | $\delta_s$ |
| Input Layer Activation Function | Tanh | Tanh | Tanh | Tanh |
| First Hidden Layer Neurons | 534 | 178 | 258 | 86 |
| First Hidden Layer Activation Function | Tanh | Tanh | Tanh | Tanh |
| Second Hidden Layer Neurons | 534 | 178 | 258 | 86 |
| Second Hidden Layer Activation Function | Tanh | Tanh | Tanh | Tanh |
| Third Hidden Layer Neurons | 534 | 178 | 258 | 86 |
| Third Hidden Layer Activation Function | Tanh | Tanh | Tanh | Tanh |
| Output Layer Neurons ($i$-th agent) | $\delta_A$ | $\delta_{A_i}$ | $\delta_A$ | $\delta_{A_i}$ |
| Output Layer Activation Function | – | – | Softmax | Softmax |



中国知网 https://www.cnki.net

**Table 3**
Factory parameters.

| Parameter | Value |
|---|---|
| $\lvert K \rvert$ | 20 |
| $\lvert I \rvert$ | 3 |
| $\lvert J \rvert$ | 5 |
| $\lvert R \rvert$ | 2 |
| $\boldsymbol{O}$ | $\begin{bmatrix} 1 & 3 & 1 \end{bmatrix}$ |
| $\boldsymbol{U}$ | $\begin{bmatrix} 50 & 50 \\ 50 & 50 \\ 50 & 50 \end{bmatrix}$ |
| $\boldsymbol{D}$ | $\begin{bmatrix} 4 & 5 & 8 & 5 & 2 \\ 5 & 6 & 12 & 3 & 6 \\ 3 & 4 & 5 & 3 & 5 \end{bmatrix}$ |
| $\boldsymbol{F}$ | $\begin{bmatrix} 20 & 20 & 20 & 20 & 20 \end{bmatrix}$ |
| $\boldsymbol{C}$ | $\begin{bmatrix} 13 & 12 \\ 6 & 7 \\ 4 & 3 \\ 2 & 4 \\ 3 & 5 \end{bmatrix}$ |
| $\boldsymbol{P}$ | $\begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ -1 & 0 & 0 & -1 & 0 \end{bmatrix}$ |
| $\boldsymbol{G}$ | $\begin{bmatrix} 1\,000 & 2\,000 \end{bmatrix}$ |

for each random state was computed and compared with the actual ending time achieved by the agent. The proportion of optimal solutions obtained without further learning is presented in Fig. 7. The PPO agent successfully provided optimal solutions in over 90% of the instances across different values of $i$ and $j$. These results highlight the potential of the proposed approach to compute on-the-fly solutions for the ALBP, even when initialized with random states. It is worth noting that the PPO agent is well-regarded for its robustness to random initialization, as demonstrated in previous works [43,44].

### 6.3. Multi-agent results

Fig. 8(a) illustrates the growth in the dimension of the action space as the number of tasks increases, for the various scheduling algorithms discussed in this paper. The figure highlights the polynomial dependence achieved by the proposed multi-agent framework, in contrast to the initial exponential growth characteristic of the single-agent framework. This result opens avenues for future research, such as further decentralizing the task assignment process, which could further reduce the dimension of the action space. The episodic ending time achieved by the multi-agent PPO during training is presented in Fig. 8(b). The shaded areas represent the standard deviation around the average ending times, computed using a moving window. The plot demonstrates that the multi-agent system converged to the optimal policy in very few iterations, despite being trained with smaller networks compared to the centralized network. As shown in the subsequent plots, the multi-agent system also achieved convergence in terms of loss during the training process. Table 4 summarizes the main training results. It is evident that the proposed multi-agent PPO significantly reduces the training time compared to both the centralized and multi-agent DQN approaches, achieving a nearly 50% reduction in training time relative to the centralized PPO.

Table 4 presents the convergence episodes and training times for all the DRL algorithms, both centralized and decentralized. Additionally, it compares the inference time (i.e., the time required to compute a solution) with the time needed to solve the same problem using the optimal control (OC) method presented in Ref. [40]. The solutions provided by the proposed DRL algorithms are more than 10 times faster than those obtained using the OC method, even in a small case study involving 3 workstations and 5 tasks.

## 7. Conclusion

This paper presented a mathematical model for a generalized industrial assembly line, based on the Markov Decision Process (MDP), which does not assume a specific assembly line type, in contrast to most existing models. This model was utilized to construct a virtual environment in which Deep Reinforcement Learning (DRL) agents are trained to optimize task and resource scheduling. Additionally, the paper introduces strategies to reduce agent training time. First, an action masking technique was implemented to ensure agents only apply feasible actions, significantly reducing training duration. Second, a multi-agent approach was developed, assigning each workstation its own agent, which promotes inter-agent cooperation and minimizes overall task completion time. This proposed multi-agent architecture offers a learning framework for optimizing industrial assembly lines, enabling agents to learn offline and subsequently deliver real-time solutions by mapping the factory's current state to optimal actions via a neural network, thus minimizing task duration. A comparative study of two Reinforcement Learning (RL) algorithms was conducted, with the introduction of action masking to improve learning efficiency and reduce training time. Moreover, the problem was decentralized into a Multi-Agent Reinforcement Learning (MARL) framework, reducing the action space's growth rate from exponential to polynomial. A sequential training algorithm was developed to ensure that selected actions remain feasible. Numerical simulations demonstrate that the proposed multi-agent framework reduces training time by nearly tenfold compared to

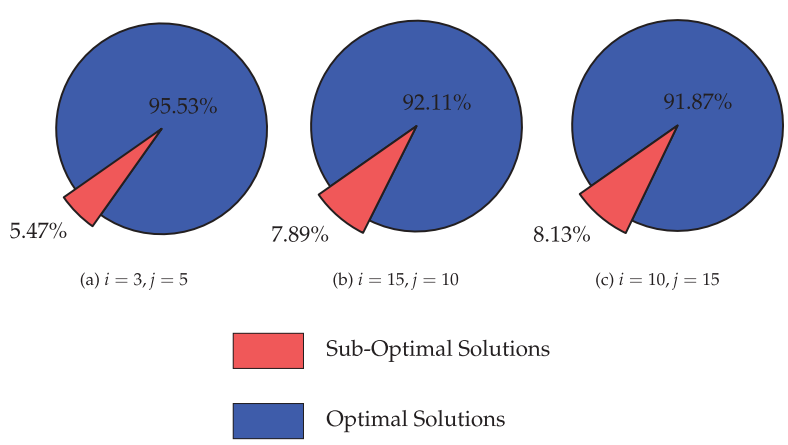


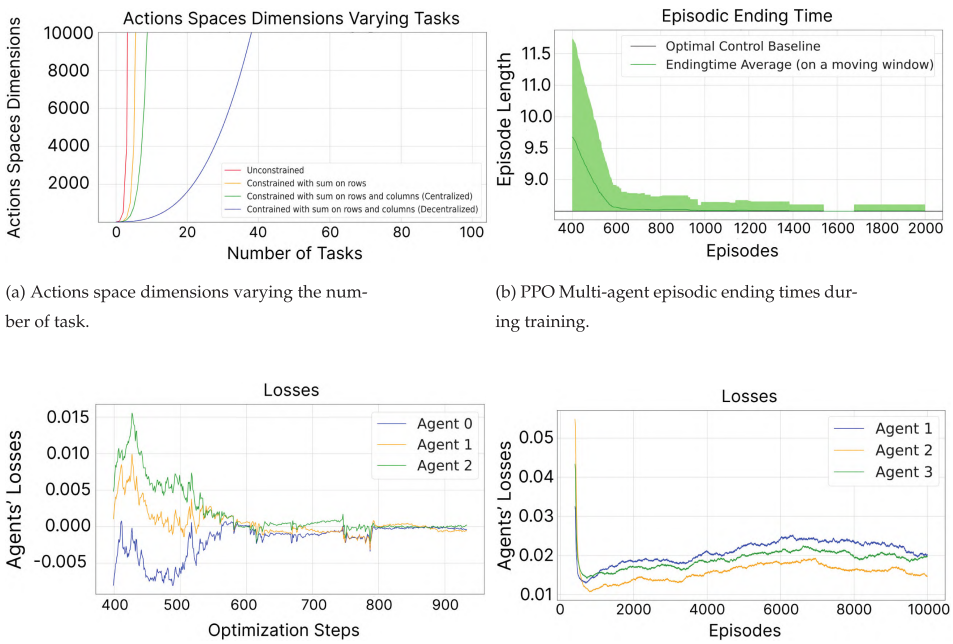

**Fig. 7.** Robustness test results.



中国知网 https://www.cnki.net

**Table 4**
Training results comparison.

| Case | Algorithm | Training time | Convergence episode | Inference time |
|---|---|---|---|---|
| 3 Workstations, 5 Tasks | DQN Centralized | 01:05:48 | 5 300 | 0.383 |
| | DQN Multi-agent | 00:51:14 | 8 600 | 0.095 |
| | PPO Centralized | 00:21:26 | 600 | 0.555 |
| | PPO Multi-agent | **00:11:14** | **800** | **0.36** |
| | Optimal Control | – | – | 3.976 |
| 15 Workstations, 10 Tasks | DQN Centralized | 03:30:48 | 6 100 | 0.698 |
| | DQN Multi-agent | 02:45:05 | 9 400 | 0.095 |
| | PPO Centralized | 01:45:00 | 740 | 0.58 |
| | PPO Multi-agent | **00:25:29** | **1 050** | **0.42** |
| | Optimal Control | – | – | 45.216 |
| 10 Workstations, 15 Tasks | DQN Centralized | 03:05:48 | 5 390 | 0.703 |
| | DQN Multi-agent | 02:48:56 | 9 600 | 0.095 |
| | PPO Centralized | 01:19:26 | 760 | 0.555 |
| | PPO Multi-agent | **00:19:14** | **820** | **0.36** |
| | Optimal Control | – | – | 45.436 |




(a) Actions space dimensions varying the number of task.

(b) PPO Multi-agent episodic ending times during training.

(c) PPO Multi-agent losses during training.

(d) DQN Multi-agent losses during training.

**Fig. 8.** Part (a) shows how the action space size increases, varying the number of tasks. Part (b) illustrates the episodic ending time achieved by centralized PPO during training. Part (c) and Part (d) show the agents' losses during training for multi-agent PPO and DQN, respectively.

a centralized framework. The Proximal Policy Optimization (PPO) algorithm exhibited superior performance, converging to optimal solutions more effectively than the traditional Deep Q-Learning algorithm. Future work will address the potential discrepancies between the simulated environment used for training the DRL agent and the actual deployment environment.

## CRediT authorship contribution statement

**Ali M. Ali:** Writing – original draft, Software, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Luca Tirel:** Writing – original draft, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Hashim A. Hashim:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This work was supported in part by the National Sciences and Engineering Research Council of Canada (NSERC) under the grants RGPIN-2022-04937.

## Data availability

No data was used for the research described in the article.



中国知网 https://www.cnki.net

## Appendix. Constraints formulation in reinforcement learning framework

The constraints used to generate the mask are defined as follows, where the round brackets denote specific elements. We referred to $z \in [1, \dots, \delta_A]$ as the index of a specific action in the actions space $A$, and $m(z)$ as the corresponding value:

1. Do not assign finished tasks:

$$if\ \boldsymbol{a}[k](i,j) = 1\ \varsigma\ \boldsymbol{f}[k](j) = 1 \longrightarrow m(z) = 0. \tag{A.1}$$

2. Do not assign a task if it is already in an execution state:

$$if\ \boldsymbol{a}[k](i,j) = 1\ \varsigma\ \sum_{i \in |I|} \boldsymbol{e}[k](i,j) = 1 \longrightarrow m(z) = 0. \tag{A.2}$$

3. Check deadlines are not violated when assigning a task:

$$\begin{aligned} &if\ \boldsymbol{a}[k](i,j) = 1 \\ &\varsigma\ k + \boldsymbol{D}(i,j) > \boldsymbol{F}(j) \longrightarrow m(z) = 0. \end{aligned}$$

4. Check the current workload for each workstation does not exceed maximum occupancy:

$$\forall i,\ if\ \boldsymbol{o}[k](i) + \sum_{j \in |J|} \boldsymbol{a}[k](i,j) > \boldsymbol{O}(i) \longrightarrow m(z) = 0. \tag{A.3}$$

5. Check that the time precedences are not violated when assigning tasks:

$$\begin{aligned} &if\ \boldsymbol{a}[k](i,j_1) = 1 \\ &\forall\ j_2 \in \boldsymbol{P}(j_1,j_2),\ if\ j_2 = -1\ \varsigma\ \boldsymbol{f}[k](j_2) = 0 \\ &\longrightarrow m(z) = 0. \end{aligned} \tag{A.4}$$

6. Resources required must not exceed the available space in workstations' buffers:

$$\begin{aligned} &\forall i, \forall r,\ if\ \boldsymbol{y}[k](i,j,r) = 1 \\ &if \sum_{j \in |J|} \boldsymbol{C}(j,r) + \sum_{j \in |J|} \boldsymbol{y}[k](i,j,r) > \boldsymbol{U}(i,r) \\ &\longrightarrow m(z) = 0. \end{aligned} \tag{A.5}$$

7. Resources needed must exist in the factory:

$$\begin{aligned} &\forall r,\ if \sum_{i \in |I|} \sum_{j \in |J|} \sum_{r \in |R|} \boldsymbol{y}[k](i,j,r) * \boldsymbol{C}(j,r) > \boldsymbol{g}[k](r) \\ &\longrightarrow m(z) = 0. \end{aligned} \tag{A.6}$$

## References

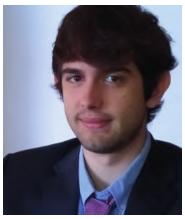

[1] P. Sivasankaran, P. Shahabudeen, Literature review of assembly line balancing problems, Int. J. Adv. Manuf. Technol. 73 (2014) 1665–1694.
[2] E. Oztemel, S. Gursev, Literature review of industry 4.0 and related technologies, J. Intell. Manuf. 31 (1) (2020) 127–182.
[3] M.E. Salveson, The assembly-line balancing problem, Trans. ASME 77 (6) (1955) 939–947.
[4] M. Cai, Z. Zhou, L. Li, S. Xiao, Z. Kan, Reinforcement learning with soft temporal logic constraints using limit-deterministic generalized Büchi automaton, J. Autom. Intell. 4 (1) (2024) 39–51.
[5] Z. Ning, L. Xie, A survey on multi-agent reinforcement learning and its application, J. Autom. Intell. 3 (2) (2024) 73–91.
[6] W. Chen, Q. Wei, A new optimal adaptive backstepping control approach for nonlinear systems under deception attacks via reinforcement learning, J. Autom. Intell. 3 (1) (2024) 34–39.
[7] B. Xiao, H.-K. Lam, X. Su, Z. Wang, F.P.-W. Lo, S. Chen, E. Yeatman, Sampled-data control through model-free reinforcement learning with effective experience replay, J. Autom. Intell. 2 (1) (2023) 20–30.
[8] J. Liu, Y. Jin, A comprehensive survey of robust deep learning in computer vision, J. Autom. Intell. 2 (4) (2023) 175–195.
[9] O. Battaïa, A. Dolgui, A taxonomy of line balancing problems and their solution approaches, Int. J. Prod. Econ. 142 (2) (2013) 259–277.
[10] I. Kucukkoc, D.Z. Zhang, Balancing of parallel U-shaped assembly lines, Comput. Oper. Res. 64 (2015) 233–244.
[11] D. Gyulai, B. Kádár, L. Monostori, Scheduling and operator control in reconfigurable assembly systems, Procedia CIRP 63 (2017) 459–464.
[12] J. Koskinen, C. Raduly-Baka, M. Johnsson, O.S. Nevalainen, Rolling horizon production scheduling of multi-model PCBs for several assembly lines, Int. J. Prod. Res. 58 (4) (2020) 1052–1073.
[13] K. Wang, X. Li, L. Gao, P. Li, S.M. Gupta, A genetic simulated annealing algorithm for parallel partial disassembly line balancing problem, Appl. Soft Comput. 107 (2021) 107404.
[14] Z.A. Çil, Z. Li, S. Mete, E. Özceylan, Mathematical model and bee algorithms for mixed-model assembly line balancing problem with physical human–robot collaboration, Appl. Soft Comput. 93 (2020) 106394.
[15] M.D. Mura, G. Dini, Designing assembly lines with humans and collaborative robots: A genetic approach, CIRP Ann. 68 (1) (2019) 1–4.
[16] J. Chen, X. Jia, Q. He, A novel bi-level multi-objective genetic algorithm for integrated assembly line balancing and part feeding problem, Int. J. Prod. Res. 61 (2) (2023) 580–603.
[17] D. Gyulai, B. Kádár, L. Monosotori, Robust production planning and capacity control for flexible assembly lines, IFAC- Pap. 48 (3) (2015) 2312–2317.
[18] M. Manzini, J. Unglert, D. Gyulai, M. Colledani, J.M. Jauregui-Becker, L. Monostori, M. Urgo, An integrated framework for design, management and operation of reconfigurable assembly systems, Omega 78 (2018) 69–84.
[19] Z. Kang, C. Catal, B. Tekinerdogan, Machine learning applications in production lines: A systematic literature review, Comput. Ind. Eng. 149 (2020) 106773.
[20] R. Buddala, S.S. Mahapatra, Two-stage teaching-learning-based optimization method for flexible job-shop scheduling under machine breakdown, Int. J. Adv. Manuf. Technol. 100 (2019) 1419–1432.
[21] L. Zhou, L. Zhang, B.K. Horn, Deep reinforcement learning-based dynamic scheduling in smart manufacturing, Procedia CIRP 93 (2020) 383–388.
[22] L. Wang, X. Hu, Y. Wang, S. Xu, S. Ma, K. Yang, Z. Liu, W. Wang, Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning, Comput. Netw. 190 (2021) 107969.
[23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv:1707.06347.
[24] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.
[25] M. Geurtsen, I. Adan, Z. Atan, Deep reinforcement learning for optimal planning of assembly line maintenance, J. Manuf. Syst. 69 (2023) 170–188.
[26] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu, Safe reinforcement learning via shielding, in: Proceedings of the AAAI Conference on Artificial Intelligence, 32, (1) 2018, pp. 2669–2678.
[27] S. Huang, S. Ontañón, A closer look at invalid action masking in policy gradient algorithms, 2020, arXiv:2006.14171.
[28] I. Elsayed-Aly, S. Bharadwaj, C. Amato, R. Ehlers, U. Topcu, L. Feng, Safe multi-agent reinforcement learning via shielding, in: Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems, 2021, pp. 483–491.
[29] P. Dai, W. Yu, H. Wang, S. Baldi, Distributed actor–critic algorithms for multiagent reinforcement learning over directed graphs, IEEE Trans. Neural Netw. Learn. Syst. 34 (10) (2022) 7210–7221.
[30] L. Buşoniu, R. Babuška, B.D. Schutter, Multi-agent reinforcement learning: An overview, Innov. Multi- Agent Syst. Appl. (2010) 183–221.
[31] A. Oroojlooy, D. Hajinezhad, A review of cooperative multi-agent deep reinforcement learning, Appl. Intell. 53 (11) (2023) 13677–13722.
[32] J.K. Gupta, M. Egorov, M. Kochenderfer, Cooperative multi-agent control using deep reinforcement learning, in: Autonomous Agents and Multiagent Systems, AAMAS 2017, Springer, 2017, pp. 66–83.
[33] T. Rashid, G. Farquhar, B. Peng, S. Whiteson, Weighted QMIX: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning, Adv. Neural Inf. Process. Syst. 33 (2020) 10199–10210.
[34] J. Wang, Z. Ren, T. Liu, Y. Yu, C. Zhang, Qplex: Duplex dueling multi-agent q-learning, 2020, arXiv:2008.01062.
[35] W. Fu, C. Yu, Z. Xu, J. Yang, Y. Wu, Revisiting some common practices in cooperative multi-agent reinforcement learning, in: Proceedings of the 39th International Conference on Machine Learning, Baltimore, Maryland, USA, 2022, pp. 6863–6877.
[36] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, Y. Wu, The surprising effectiveness of ppo in cooperative multi-agent games, Adv. Neural Inf. Process. Syst. 35 (2022) 24611–24624.
[37] B. Peng, T. Rashid, C. Schroeder de Witt, P.-A. Kamienny, P. Torr, W. Böhmer, S. Whiteson, Facmac: Factored multi-agent centralised policy gradients, Adv. Neural Inf. Process. Syst. 34 (2021) 12208–12221.
[38] M. Geurtsen, I. Adan, Z. Atan, Deep reinforcement learning for optimal planning of assembly line maintenance, J. Manuf. Syst. 69 (2023) 170–188.
[39] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, 2013, arXiv: 1312.5602.
[40] A.M. Ali, L. Tirel, Action masked deep reinforcement learning for controlling industrial assembly lines, in: 2023 IEEE World AI IoT Congress (AIIoT), 2023, pp. 0797–0803.




中国知网 https://www.cnki.net

[41] A. Kopic, E. Perenda, H. Gacanin, A collaborative multi-agent deep reinforcement learning-based wireless power allocation with centralized training and decentralized execution, IEEE Trans. Commun. 72 (11) (2024) 7006–7016.
[42] L. Zhang, Y.-C. Liang, Deep reinforcement learning for multi-agent power control in heterogeneous networks, IEEE Trans. Wirel. Commun. 20 (4) (2020) 2551–2564.
[43] Y. Emami, H. Gao, K. Li, L. Almeida, E. Tovar, Z. Han, Age of information minimization using multi-agent UAVs based on AI-enhanced mean field resource allocation, IEEE Trans. Veh. Technol. 73 (9) (2024) 13368–13380.
[44] A.M. Ali, A. Gupta, H.A. Hashim, Deep reinforcement learning for sim-to-real policy transfer of VTOL-UAVs offshore docking operations, Appl. Soft Comput. 162 (2024) 111843.

**Ali M. Ali** received the M.Sc. degree in Control Engineering from the Department of Computer, Control, and Management Engineering (DIAG), Sapienza University of Rome, Rome, Italy. Since 2023, he has been pursuing the Ph.D. degree with the Department of Mechanical and Aerospace Engineering, Carleton University, Ottawa, ON, Canada. His current research interests include nonlinear control, reinforcement learning, and applied AI.



**Luca Tirel** received the B.Sc. degree in Management Engineering and the M.Sc. degree in Control Engineering from the Department of Computer, Control, and Management Engineering (DIAG), University of Rome 'La Sapienza', Rome, Italy. He is now working as an Avionic Systems Engineer at Leonardo S.p.A., Italy. His current research interests include deep reinforcement learning and applied AI.

**Hashim A. Hashim** received the B.Sc. degree in Mechatronics from the Department of Mechanical Engineering, Helwan University, Cairo, Egypt, the M.Sc. degree in Systems and Control Engineering from the Department of Systems Engineering, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia, and the Ph.D. degree in Robotics and Control from the Department of Electrical and Computer Engineering, Western University, ON, Canada. He is currently an Assistant Professor with the Department of Mechanical and Aerospace Engineering, Carleton University, Ottawa, ON. From 2019 to 2021, he was an Assistant Professor with the Department of Engineering and Applied Science, Thompson Rivers University, Kamloops, BC, Canada. His current research interests include vision-based aided navigation and control, localization and mapping, stochastic and deterministic estimation, sensor fusion, and distributed control of multi-agent systems.



中国知网 https://www.cnki.net